AWS システム設計資料 - ファイル共有・ストレージシステム

システム概要

S3 をベースに、セキュアなファイル保管と共有機能を提供するシステム。署名付き URL、アクセス制御、バージョニングにより、エンタープライズグレードのストレージソリューションを実現します。

対象用途:

- 社内ファイル共有
- ドキュメント管理
- バックアップストレージ
- メディアライブラリ
- ユーザー生成コンテンツ保管

アーキテクチャ構成

```
ユーザー (Web/Mobile/Desktop)
    ↓ HTTPS
  API Gateway
    ↓
Lambda Function
├─ アップロード処理
├─ ダウンロード処理
├─ ファイル共有管理
└─ アクセス制御
    ↓
S3 Bucket
├─ ファイルストレージ
├─ バージョニング
├─ 暗号化
└─ ライフサイクル管理
    ↓
DynamoDB
└─ ファイルメタデータ
    (ファイル名、所有者、共有設定、タイムスタンプ)
    ↓
CloudWatch
└─ 監視・ログ記録
```

使用するAWSサービス

サービス	役割	責務内容
S3	ファイルストレージ	ファイル保存、バージョン管理、暗号化

サービス	役割	責務内容
API Gateway	ファイル操作 API	アップロード/ダウンロードエンドポイント
Lambda	ビジネスロジック	ファイル処理、権限管理、メタデータ更新
DynamoDB	メタデータDB	ファイル情報、ユーザー設定、共有情報
IAM	アクセス制御	ユーザー認証、権限管理
CloudTrail	監査ログ	すべてのアクセス記録

詳細設定値

S3 バケット設定

```
バケット名: company-file-storage
リージョン: ap-northeast-1

バージョニング:
  - 状態: 有効
  - 理由: ファイル復元、変更履歴管理

サーバー側暗号化 (SSE-S3):
  - 有効: ○
  - アルゴリズム: AES-256
  - マスターキー管理: Amazon S3 管理

ブロック公開アクセス設定:
  - BlockPublicAcls: 有効
  - IgnorePublicAcls: 有効
  - BlockPublicPolicy: 有効
  - RestrictPublicBuckets: 有効

アクセスログ:
  - 有効: ○
  - ログ保存先: s3://access-logs-bucket/s3-logs/
  - ログ対象: すべてのリクエスト

オブジェクトロック:
  - 有効: ○ (重要データ向け)
  - モード: Compliance
  - 保持期間: 90日 (カスタマイズ可)

ライフサイクルルール:

  1. ルール名: "Archive to Glacier"
     - 対象: すべてのオブジェクト
     - 条件: 90日経過後
     - アクション: Glacier へ転送
     - 非現在版: 30日後に Glacier へ転送

  2. ルール名: "Delete Old Versions"
```

```
      - 対象: 非現在版
      - 条件: 365日経過後
      - アクション: 削除

    3. ルール名: "Delete Incomplete Uploads"
      - 対象: マルチパートアップロード断片
      - 条件: 7日経過
      - アクション: 削除

  CORS (クロスオリジンリソース共有):
    - 有効: ○
    - 許可オリジン:
      * https://app.example.com
      * https://admin.example.com
    - 許可メソッド: GET, PUT, POST, DELETE, HEAD
    - 許可ヘッダー:
      * Content-Type
      * Authorization
      * x-amz-*
    - 公開ヘッダー:
      * x-amz-version-id
      * ETag

  静的ホスティング:
    - 有効: 無効（API 経由でのみアクセス）

  タグ付け:
    - Department: IT
    - CostCenter: Shared
    - DataClassification: Internal
    - BackupRequired: Yes
```

S3 フォルダ構成例

```
company-file-storage/
├── users/
│   ├── user-001/
│   │   ├── documents/
│   │   ├── uploads/
│   │   └── shared-with-me/
│   └── user-002/
├── shared/
│   ├── department-a/
│   ├── department-b/
│   └── organization-wide/
├── archives/
│   ├── 2024-q1/
│   ├── 2024-q2/
│   └── 2024-q3/
└── temp/
    └── processing/
```

DynamoDB メタデータテーブル設定

```
テーブル名: FileMetadata

キャパシティモード: オンデマンド

パーティションキー:
  - 属性名: fileId
  - 型: 文字列 (S)

ソートキー: なし

属性定義:

  - fileId (S): ファイル一意識別子
  - ownerId (S): 所有者ユーザーID
  - fileName (S): ファイル名
  - fileSize (N): ファイルサイズ（バイト）
  - fileType (S): MIME タイプ
  - s3Key (S): S3 キー（パス）
  - s3VersionId (S): S3 バージョンID
  - uploadedAt (S): アップロード日時 ( ISO 8601)
  - modifiedAt (S): 更新日時
  - expiresAt (S): 有効期限 ( TTL)
  - status (S): アップロード状態 ( pending, completed, failed)
  - checksum (S): SHA-256 ハッシュ値
  - tags (M): ファイルタグ
  - shareSettings (M): 共有設定 ( 詳細下記 )
  - accessLog (L): アクセスログリスト

グローバルセカンダリインデックス (GSI):

  1. インデックス名: OwnerIdIndex
     - パーティションキー: ownerId
     - ソートキー: uploadedAt
     - 投影タイプ: ALL
     - 用途: ユーザーごとのファイル一覧

  2. インデックス名: FileNameIndex
     - パーティションキー: fileName
     - ソートキー: uploadedAt
     - 投影タイプ: KEYS_ONLY
     - 用途: ファイル名検索

  3. インデックス名: StatusIndex
     - パーティションキー: status
     - ソートキー: uploadedAt
     - 投影タイプ: ALL
     - 用途: アップロード進捗確認

TTL (Time To Live):
```

```
  - 属性名: expiresAt
  - 有効: ○
  - 用途: 一時ファイルの自動削除

ストリーム:
  - 有効: ○
  - ビュータイプ: NEW_AND_OLD_IMAGES
  - 用途: メタデータ変更の監視

DynamoDB アイテム例：

```json
{
  "fileId": "file-uuid-12345",
  "ownerId": "user-001",
  "fileName": "proposal.pdf",
  "fileSize": 2048576,
  "fileType": "application/pdf",
  "s3Key": "users/user-001/documents/proposal.pdf",
  "s3VersionId": "xyz789abc123",
  "uploadedAt": "2024-11-02T10:30:00Z",
  "modifiedAt": "2024-11-02T10:30:00Z",
  "status": "completed",
  "checksum": "sha256:abc123def456...",
  "tags": {
    "project": "q4-planning",
    "priority": "high",
    "department": "sales"
  },
  "shareSettings": {
    "isPublic": false,
    "sharedWith": [
      {
        "userId": "user-002",
        "permission": "read",
        "sharedAt": "2024-11-01T15:00:00Z"
      },
      {
        "userId": "user-003",
        "permission": "read-write",
        "sharedAt": "2024-11-01T14:00:00Z"
      }
    ],
    "externalLinks": [
      {
        "linkId": "link-uuid-xyz",
        "expiresAt": "2024-12-02T00:00:00Z",
        "permission": "read",
        "password": "hashed-password",
        "downloadCount": 5,
        "maxDownloads": 10
      }
    ]
  },
```
```

```
  "accessLog": [
    {
      "userId": "user-002",
      "action": "download",
      "timestamp": "2024-11-02T09:15:00Z",
      "ipAddress": "203.0.113.25"
    }
  ]
}
```

Lambda 関数設定

```
関数名: file-management-service

実行時: Python 3.11

基本設定:
  - メモリ: 1024 MB（大容量ファイル処理）
  - タイムアウト: 300秒（5分）
  - 一時ストレージ (/tmp): 10,240 MB

環境変数:
  - S3_BUCKET: company-file-storage
  - DYNAMODB_TABLE: FileMetadata
  - MAX_FILE_SIZE: 5368709120 (5GB)
  - ALLOWED_EXTENSIONS: pdf,doc,docx,xlsx,pptx,jpg,png
  - VIRUS_SCAN_ENABLED: true
  - ENCRYPTION_KEY_ID: arn:aws:kms:...

IAM 実行ロール:
  - s3:GetObject
  - s3:PutObject
  - s3:DeleteObject
  - s3:ListBucket
  - dynamodb:GetItem
  - dynamodb:PutItem
  - dynamodb:UpdateItem
  - dynamodb:Query
  - kms:Decrypt
  - kms:GenerateDataKey
  - logs:CreateLogGroup
  - logs:CreateLogStream
  - logs:PutLogEvents

レイヤー:
  - 依存関係: boto3, requests, python-magic
  - 共有ユーティリティ: 暗号化、ハッシング、ファイル検証
```

Lambda 関数コード例 (Python)

```python
import json
import boto3
import hashlib
import uuid
import os
from datetime import datetime, timedelta
from urllib.parse import quote

s3 = boto3.client('s3')
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table(os.environ['DYNAMODB_TABLE'])

BUCKET_NAME = os.environ['S3_BUCKET']
MAX_FILE_SIZE = int(os.environ.get('MAX_FILE_SIZE', 5368709120))

def lambda_handler(event, context):
    """メインハンドラー"""
    http_method = event.get('httpMethod')
    path = event.get('path')

    try:
        if http_method == 'POST' and '/upload' in path:
            return handle_upload_init(event)
        elif http_method == 'PUT' and '/upload' in path:
            return handle_upload_chunk(event)
        elif http_method == 'GET' and '/download' in path:
            return handle_download(event)
        elif http_method == 'GET' and '/list' in path:
            return handle_list_files(event)
        elif http_method == 'DELETE' and '/file' in path:
            return handle_delete_file(event)
        elif http_method == 'PUT' and '/share' in path:
            return handle_share_file(event)
        else:
            return error_response(404, 'Not Found')

    except Exception as e:
        return error_response(500, f'Error: {str(e)}')

def handle_upload_init(event):
    """アップロード初期化"""
    user_id = get_user_id(event)
    body = json.loads(event.get('body', '{}'))

    file_name = body.get('fileName')
    file_size = body.get('fileSize')
    file_type = body.get('fileType')

    # バリデーション
    if not file_name or not file_size or file_size > MAX_FILE_SIZE:
        return error_response(400, 'Invalid file parameters')

    # ファイルID生成
```

```
    file_id = str(uuid.uuid4())

    # S3 マルチパートアップロード初期化
    s3_key = f'users/{user_id}/documents/{file_id}-{file_name}'

    try:
        mpu = s3.create_multipart_upload(
            Bucket=BUCKET_NAME,
            Key=s3_key,
            ContentType=file_type,
            ServerSideEncryption='AES256',
            Metadata={
                'fileId': file_id,
                'userId': user_id,
                'originalName': file_name
            }
        )

        upload_id = mpu['UploadId']

        # メタデータを DynamoDB に保存
        table.put_item(Item={
            'fileId': file_id,
            'ownerId': user_id,
            'fileName': file_name,
            'fileSize': file_size,
            'fileType': file_type,
            's3Key': s3_key,
            'uploadedAt': datetime.utcnow().isoformat(),
            'status': 'pending',
            'uploadId': upload_id,
            'tags': {},
            'shareSettings': {'isPublic': False, 'sharedWith': []}
        })

        return success_response(200, {
            'fileId': file_id,
            'uploadId': upload_id,
            's3Key': s3_key
        })

    except Exception as e:
        return error_response(500, f'Upload initialization failed: {str(e)}')

def handle_download(event):
    """ダウンロード処理（署名付きURL生成）"""
    user_id = get_user_id(event)
    file_id = event.get('queryStringParameters', {}).get('fileId')

    # ファイルメタデータ取得
    response = table.get_item(Key={'fileId': file_id})

    if 'Item' not in response:
        return error_response(404, 'File not found')
```

```
    file_data = response['Item']

    # アクセス権限確認
    if not check_permission(user_id, file_data, 'read'):
        return error_response(403, 'Access denied')

    # 署名付き URL 生成
    s3_key = file_data['s3Key']

    try:
        url = s3.generate_presigned_url(
            'get_object',
            Params={
                'Bucket': BUCKET_NAME,
                'Key': s3_key,
                'ResponseContentDisposition': f"attachment; filename=
{quote(file_data['fileName'])}"
            },
            ExpiresIn=3600  # 1時間有効
        )

        # アクセスログ記録
        log_access(file_id, user_id, 'download')

        return success_response(200, {'downloadUrl': url})

    except Exception as e:
        return error_response(500, f'Download URL generation failed: {str(e)}')

def handle_list_files(event):
    """ファイル一覧取得"""
    user_id = get_user_id(event)

    try:
        # ユーザーが所有するファイル
        response = table.query(
            IndexName='OwnerIdIndex',
            KeyConditionExpression='ownerId = :uid',
            ExpressionAttributeValues={':uid': user_id},
            Limit=100
        )

        files = []
        for item in response.get('Items', []):
            files.append({
                'fileId': item['fileId'],
                'fileName': item['fileName'],
                'fileSize': item['fileSize'],
                'uploadedAt': item['uploadedAt'],
                'status': item['status'],
                'sharedWith': len(item.get('shareSettings', {}).get('sharedWith',
[]))
            })
```

```
        return success_response(200, {'files': files})

    except Exception as e:
        return error_response(500, f'List operation failed: {str(e)}')

def handle_delete_file(event):
    """ファイル削除"""
    user_id = get_user_id(event)
    file_id = event.get('queryStringParameters', {}).get('fileId')

    response = table.get_item(Key={'fileId': file_id})

    if 'Item' not in response:
        return error_response(404, 'File not found')

    file_data = response['Item']

    # オーナー確認
    if file_data['ownerId'] != user_id:
        return error_response(403, 'Only owner can delete')

    try:
        # S3 から削除
        s3.delete_object(Bucket=BUCKET_NAME, Key=file_data['s3Key'])

        # DynamoDB から削除
        table.delete_item(Key={'fileId': file_id})

        return success_response(204, None)

    except Exception as e:
        return error_response(500, f'Delete failed: {str(e)}')

def handle_share_file(event):
    """ファイル共有設定"""
    user_id = get_user_id(event)
    file_id = event.get('queryStringParameters', {}).get('fileId')
    body = json.loads(event.get('body', '{}'))

    response = table.get_item(Key={'fileId': file_id})

    if 'Item' not in response:
        return error_response(404, 'File not found')

    file_data = response['Item']

    # オーナー確認
    if file_data['ownerId'] != user_id:
        return error_response(403, 'Only owner can share')

    # 共有ユーザーを追加
    shared_user_id = body.get('userId')
    permission = body.get('permission', 'read')  # read or read-write
```

```
    share_settings = file_data.get('shareSettings', {'isPublic': False,
'sharedWith': []})

    share_settings['sharedWith'].append({
        'userId': shared_user_id,
        'permission': permission,
        'sharedAt': datetime.utcnow().isoformat()
    })

    try:
        table.update_item(
            Key={'fileId': file_id},
            UpdateExpression='SET shareSettings = :ss',
            ExpressionAttributeValues={':ss': share_settings}
        )

        return success_response(200, {'message': 'File shared successfully'})

    except Exception as e:
        return error_response(500, f'Share failed: {str(e)}')

def check_permission(user_id, file_data, action):
    """アクセス権限確認"""
    # オーナー
    if file_data['ownerId'] == user_id:
        return True

    # 共有ユーザー
    share_settings = file_data.get('shareSettings', {})

    for shared in share_settings.get('sharedWith', []):
        if shared['userId'] == user_id:
            if action == 'read':
                return True
            elif action == 'write' and shared['permission'] == 'read-write':
                return True

    return False

def log_access(file_id, user_id, action):
    """アクセスログ記録"""
    table.update_item(
        Key={'fileId': file_id},
        UpdateExpression='SET accessLog = list_append(accessLog, :log)',
        ExpressionAttributeValues={
            ':log': [{
                'userId': user_id,
                'action': action,
                'timestamp': datetime.utcnow().isoformat()
            }]
        }
    )
```

```python
def get_user_id(event):
    """リクエストからユーザーID取得"""
    claims = event.get('requestContext', {}).get('authorizer', {})
    return claims.get('claims', {}).get('sub')

def success_response(status_code, body):
    return {
        'statusCode': status_code,
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*'
        },
        'body': json.dumps(body) if body else ''
    }

def error_response(status_code, message):
    return {
        'statusCode': status_code,
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*'
        },
        'body': json.dumps({'error': message})
    }
```

API Gateway リソース設定

```
API 名: file-management-api

エンドポイント:

1. POST /upload
   - 説明: アップロード初期化
   - Lambda: file-management-service
   - 認証: Cognito
   - リクエスト体:
     {
       "fileName": "document.pdf",
       "fileSize": 2048576,
       "fileType": "application/pdf"
     }

2. PUT /upload/{uploadId}/{partNumber}
   - 説明: マルチパートアップロード
   - Lambda: file-management-service

3. GET /download?fileId={fileId}
   - 説明: ダウンロード（署名付きURL）
   - Lambda: file-management-service
   - キャッシュ: 無効
```

```
4. GET /list
   - 説明: ファイル一覧
   - Lambda: file-management-service
   - キャッシュ: TTL 60秒

5. DELETE /file?fileId={fileId}
   - 説明: ファイル削除
   - Lambda: file-management-service

6. PUT /share?fileId={fileId}
   - 説明: ファイル共有設定
   - Lambda: file-management-service
   - リクエスト体:
     {
       "userId": "user-002",
       "permission": "read"
     }
```

推定月額コスト

シナリオ: 1000ユーザー、月間50GBアップロード、月間200GBダウンロード

項目	使用量	単価	月額費用
S3 ストレージ	500 GB	$0.023/GB	$11.50
S3 PUT リクエスト	1000万	$0.005/1000	$50.00
S3 GET リクエスト	5000万	$0.0004/1000	$20.00
S3 マルチパートアップロード	1000万チャンク	$0.005/1000	$50.00
Lambda 実行時間	50,000,000 GB-秒	$0.0000166667	$833.33
DynamoDB 読み込み	500万 RCU	$1.25/100万	$6.25
DynamoDB 書き込み	100万 WCU	$6.25/100万	$6.25
CloudTrail	監査ログ	$2.00	$2.00
合計			**$979.33**

セキュリティ設定

バケットポリシー

```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyUnencryptedObjectUploads",
      "Effect": "Deny",
      "Principal": "*",
```

```
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::company-file-storage/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption": "AES256"
        }
      }
    },
    {
      "Sid": "DenyInsecureTransport",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::company-file-storage",
        "arn:aws:s3:::company-file-storage/*"
      ],
      "Condition": {
        "Bool": {
          "aws:SecureTransport": "false"
        }
      }
    }
  ]
}
```

暗号化設定

```
転送中: TLS 1.2 以上（強制）
保存時: AES-256 (SSE-S3)
キー管理: AWS管理キー（デフォルト）
オプション: KMS カスタマーマスターキー（CMK）
```